

Tiny URL - Resumen de diseño

1. Objetivo

El proyecto implementa una aplicación de URLs cortas que permite:

- crear una URL corta con un código generado o un alias personalizado;
- resolver el código y redirigir a la URL original;
- cachear resoluciones frecuentes;
- registrar cada acceso de forma asincrónica;
- consultar la cantidad total de clicks y la fecha del último acceso.

La solución prioriza separación de responsabilidades, tolerancia a fallos parciales y un alcance acotado al challenge.

2. Arquitectura



La API y el worker son procesos separados, pero comparten el mismo código, configuración y contrato de eventos.

El frontend usa React y Tailwind CSS. Vite lo sirve en desarrollo y genera los assets estáticos que Express expone en producción.

Dentro de cada funcionalidad HTTP se sigue esta dirección:

```
route -> controller -> service -> repository -> model -> MongoDB
```

- **Routes:** definen métodos y paths.
- **Controllers:** traducen requests y responses HTTP.
- **Services:** contienen validaciones y reglas de negocio.
- **Repositories:** encapsulan la persistencia y traducen errores de MongoDB.

- **Models:** definen documentos e índices de Mongoose.
- **Infrastructure:** administra conexiones con MongoDB, Redis y RabbitMQ.

3. Datos

ShortUrl

Se almacena en la colección `shorturls`:

```
{
  code: string;
  originalUrl: string;
  createdAt: Date;
  updatedAt: Date;
}
```

`code` tiene un índice único. MongoDB es quien garantiza definitivamente que no existan dos URLs con el mismo código.

ClickEvent

Cada acceso se almacena en `click_events`:

```
{
  eventId: string;
  code: string;
  occurredAt: Date;
  ip?: string;
  userAgent?: string;
}
```

`eventId` tiene un índice único para que el procesamiento sea idempotente. Si RabbitMQ entrega dos veces el mismo evento, MongoDB impide que se persista dos veces.

Para simplificar el challenge, las estadísticas cuentan eventos individuales. En un sistema de alto volumen convendría mantener también un agregado por código con `totalClicks` y `lastClick`.

4. Flujos principales

Creación

1. Zod valida la URL y el alias opcional.
2. El service genera un código cuando no se envía un alias.
3. El repository persiste el documento en MongoDB.
4. Una colisión de código se traduce a un error propio de la aplicación.

Los códigos generados usan `randomBytes(6).toString("base64url")` y se reintentan como máximo cinco veces ante una colisión.

El alias `health` está reservado porque corresponde al endpoint operativo de la API.

Resolución

1. Se valida el código.
2. Se busca `short-url:{code}` en Redis.
3. Ante un cache miss, se consulta MongoDB y se carga Redis con TTL.
4. La API publica un evento de acceso en RabbitMQ.
5. Se responde con un redirect `302`.

MongoDB es la fuente de verdad. Si Redis no está disponible, la resolución continúa contra MongoDB. Si RabbitMQ falla, el redirect también continúa, aunque ese acceso puede no quedar registrado. La espera de confirmación del broker tiene un timeout configurable para no dejar abierto el redirect indefinidamente.

Procesamiento de accesos

El worker consume `tinyurl.accessed.v1`, valida el mensaje y lo persiste:

- evento persistido o duplicado: `ack`;
- mensaje inválido: `reject` sin requeue;
- error transitorio de persistencia: `nack` con requeue.

`prefetch(10)` limita a diez la cantidad de mensajes sin confirmar que puede recibir cada instancia del worker.

5. Por qué RabbitMQ

RabbitMQ se usa porque el problema requiere una cola de trabajo tradicional: desacoplar el redirect del registro del click y entregar cada evento a un worker para su procesamiento.

Para este caso ofrece:

- acknowledgements explícitos;
- reentrega ante fallos;
- colas y mensajes durables;
- control de concurrencia mediante `prefetch`;
- ejecución local sencilla con Docker Compose;
- una topología pequeña y fácil de inspeccionar.

También permite mantener Redis dedicado al cache, en lugar de usarlo simultáneamente como almacenamiento de mensajes.

Comparación con SQS

Amazon SQS sería una alternativa válida, especialmente si el sistema se desplegara en AWS:

- es administrado y reduce la operación del broker;
- escala automáticamente;
- ofrece colas estándar y dead-letter queues;
- su modelo de entrega al menos una vez encaja con el consumidor idempotente.

No se eligió porque agregaría una dependencia de AWS y haría menos autocontenido el entorno local del assessment. En una arquitectura desplegada en AWS, SQS probablemente sería una elección igual o más conveniente que administrar RabbitMQ.

Comparación con Kafka

Kafka está orientado a un log distribuido de eventos, con retención, replay, particiones y múltiples grupos de consumidores. Es una buena opción cuando varios sistemas necesitan consumir el historial de clicks, reconstruir estado o procesar un volumen muy alto.

Este proyecto solo necesita transportar un evento desde la API hacia un worker. No requiere replay, múltiples consumidores independientes ni procesamiento de streams. Por eso Kafka agregaría complejidad operativa y conceptual sin aportar un beneficio proporcional al caso de uso.

La decisión no implica que RabbitMQ sea universalmente superior:

- **RabbitMQ:** adecuado para esta cola de trabajo y para un entorno autocontenido.
- **SQS:** adecuado para una solución administrada dentro de AWS.
- **Kafka:** adecuado si los accesos se convierten en un stream compartido y reutilizable por varios consumidores.

6. Errores y observabilidad

Zod valida entradas en tiempo de ejecución. Los errores esperados se representan con `AppError`, mientras que el middleware global `errorHandler` produce una respuesta HTTP consistente:

```
{
  "error": {
    "code": "MACHINE_READABLE_CODE",
    "message": "Human-readable message"
  }
}
```

Los controllers no serializan errores manualmente. Express 5 propaga las promesas rechazadas hasta el middleware global.

JSON malformado, payloads excedidos y rutas inexistentes también se traducen al mismo envelope con los códigos `INVALID_JSON`, `PAYLOAD_TOO_LARGE` y `ROUTE_NOT_FOUND`.

Pino y `pino-http` generan logs estructurados y asignan un `x-request-id`. Los fallos esperados no se registran repetidamente en cada capa, y los errores desconocidos se informan en los límites HTTP o de proceso.

7. Decisiones y limitaciones

- MongoDB es la fuente de verdad.
- Redis se usa únicamente como cache.
- RabbitMQ desacopla el redirect del procesamiento del click.
- El worker es idempotente mediante `eventId`.

- Se usa `302` para que los accesos futuros vuelvan a pasar por la aplicación.
- Docker Compose permite ejecutar todo el sistema localmente.
- No se incluyen usuarios, autenticación, edición ni analytics avanzados.
- No existe una transacción entre el redirect y la publicación del evento.
- No se implementaron transactional outbox ni dead-letter queue.
- Las estadísticas actuales priorizan simplicidad sobre escalabilidad.

El diseño completo y el detalle de implementación se encuentran en `docs/design-doc.md` y `docs/entrega-tecnica.md`.